Sardar Patel Institute of Technology

Department of Computer Engineering

Academic Year 2026-27

Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Extraction and Natural-Language Queries

Shivam Mulani, Gurshaan Nandrajog, Harshvarddhan Waghela, B.Tech.(Comp.)

Project Guide: Prof. Anant Nimkar | Group No. 86

Abstract

College timetables are published as dense multi-page PDFs that everyone reads by hand. We built Dynamic Timetable, a role-based mobile and web system that extracts the official timetable automatically (PDF, scans, Excel, Word), flags uncertain entries, detects clashes and answers plain-English questions through a safe rule-based engine. On the college's real 10-page timetable, 452 of 453 classes were verified automatically, exactly 2 genuine clashes were found, and a search takes about 40 ms.

Introduction

The department publishes its class timetable as a 10-page PDF: grids per division with stacked lab batches (A1-A4), teacher codes and page-wise legends. Everyone reads it by hand many times a day, and clashes are noticed only when they happen.

Objective(s)

- Read the real timetable automatically; flag every uncertain value
- Role-based views for Student, Professor, HOD, Principal, Admin
- Answer plain-English questions safely; detect clashes
- One app for Android, iPhone and laptop; free online demo

Problem Definition

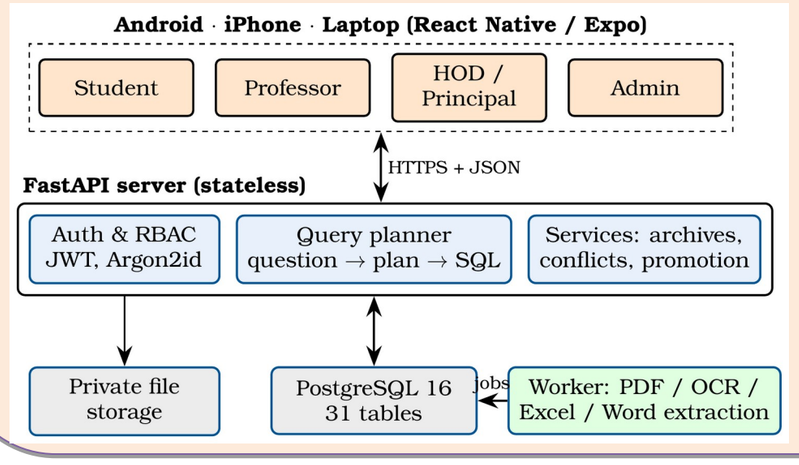
Turn a human-formatted timetable (merged and stacked cells, missing a.m./p.m., legends, scans) into verified data, and give each user correct, role-scoped answers without guessing and without exposing personal data.

Contribution

- Timetable-aware parser (backtracking time-row resolver, lab groups, legends)
- Clash detector: 542 raw overlaps → 2 genuine clashes
- Compositional query planner: question → typed plan → safe SQL (no fixed question list)

Design

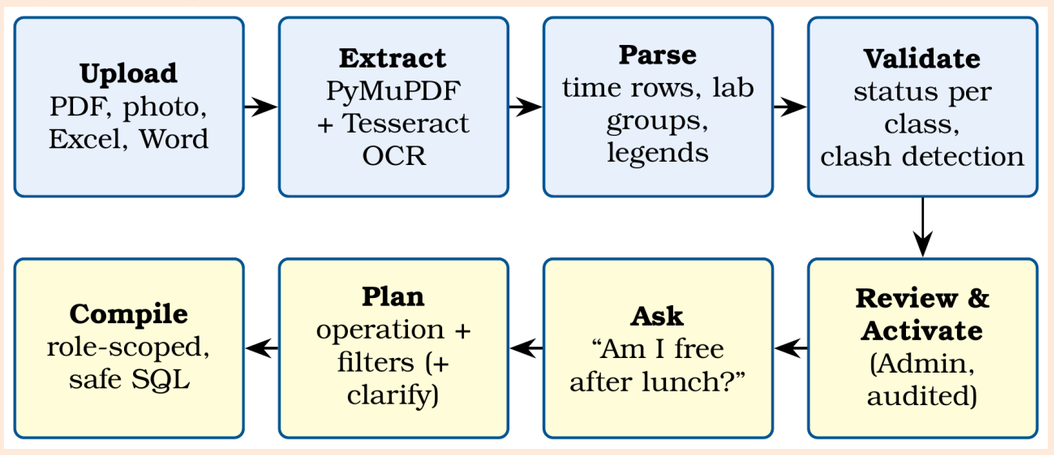
React Native app → stateless FastAPI server → PostgreSQL, plus a background extraction worker.



Scan: source code

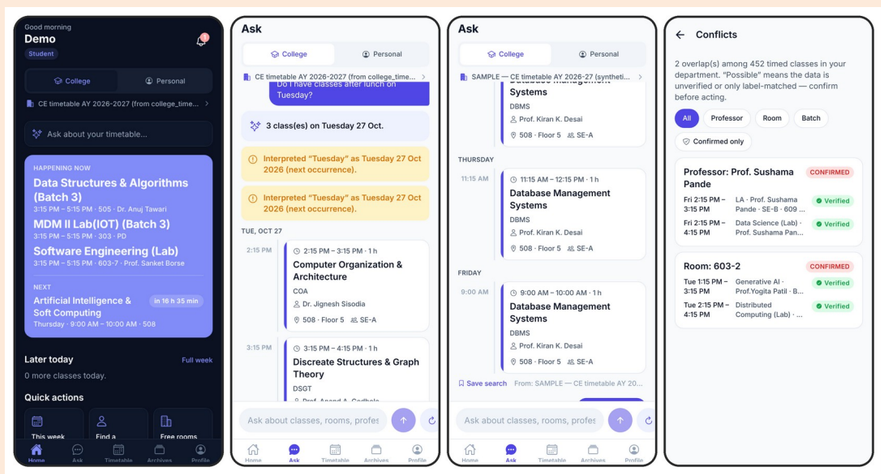
Methodology/Algorithms

1. Upload: text with coordinates (PyMuPDF), or OCR (Tesseract) for scans and photos.
2. Time rows resolved by constrained backtracking; only a unique valid reading is accepted.
3. Cells split into lab groups, legend codes expanded; every class gets a status.
4. Clashes: group by teacher/room/batch, sort, sweep; overlap if $s1 < e2$ and $s2 < e1$.
5. Question → query plan (count/first/last/per-day + filters) → clarify if ambiguous → safe SQL.



Results

- Real PDF: 453 classes, 452 verified (99.8%); the remaining one has no time printed in the PDF
- Exactly 2 genuine clashes reported (a teacher on Friday, room 603-2 on Tuesday)
- Search 37-47 ms (median); 0 wrong answers with 32 concurrent users
- 10-page PDF extracted in 1.65 s; 174 + 20 automated tests and end-to-end tests pass



Conclusion

Dynamic Timetable turns the published PDF into a trustworthy, searchable app for every role. Showing uncertainty instead of guessing kept every automatic answer correct, and the clash report found real errors in the official timetable.

References

[1] R. Zanibbi et al., "A survey of table recognition," IJDAR, 2004.

[2] R. Smith, "An overview of the Tesseract OCR engine," ICDAR, 2007.

[3] I. Androutsopoulos et al., "Natural language interfaces to databases," Nat. Lang. Eng., 1995.